

Efficient matrix inversion based on VLIW architecture

Li Zhang, Fu Li*, and Guangming Shi

School of Electronic Engineering, Xidian University, Xi'an 710071, China

Abstract: Matrix inversion is a critical part in communication, signal processing and electromagnetic system. A flexible and scalable very long instruction word (VLIW) processor with clustered architecture is proposed for matrix inversion. A global register file (RF) is used to connect all the clusters. Two nearby clusters share a local register file. The instruction sets are also designed for the VLIW processor. Experimental results show that the proposed VLIW architecture takes only 45 latency to invert a 4×4 matrix when running at 150 MHz. The proposed design is roughly five times faster than the DSP solution in processing speed.

Keywords: matrix inversion, very long instruction word (VLIW), latency, register file (RF), cluster.

DOI: 10.1109/JSEE.2014.00045

1. Introduction

As a novel technology, orthogonal frequency division multiplexing (OFDM) is widely used in wireless communication systems because of its high spectral efficiency with low computational complexity. In conjunction with it, the multiple input multiple output (MIMO) systems can further improve the capacity and performance of channels. Matrix inversion is a common operation for various algorithms that are employed in MIMO-OFDM systems, such as the zero-forcing (ZF) and minimum mean square error (MMSE) equalization.

However, matrix inversion is still a computationally intensive task in many real-time systems. For decades, researchers have made great efforts on matrix inversion algorithms and hardware implementation [1–5]. Some of the most prominent algorithms today are direct analytic matrix inversion [1], QR decomposition [2,5–8] and blockwise matrix inversion [1,4]. These algorithms have different implementation complexities when inverting matrices of different scales. The complexity of the direct analytic method increases greatly when the matrix dimension

grows. The QR decomposition algorithm is suitable for the matrix with large dimensions. The analytic algorithm can be used for small matrix inversion, comparing with the QR decomposition algorithm. In general, the complexity of matrix inversion is proportional to N^3 , where N is the size of the matrix [3].

Currently, there are mainly two kinds of hardware implementation for matrix inversion. One is the field programmable gate array (FPGA) with state machine architecture. It can achieve a high throughput, but lacks of certain flexibility [5]. This means we have to re-design the whole circuits when the specifications change. The other traditional method for hardware implementation is based on the digital signal processing (DSP) platform [9]. It can accomplish the inversion by software programming strategy. It is more flexible than the FPGA platform. However, the features of DSP limit its use in the real-time applications where parallelism and high throughput are required.

The very long instruction word (VLIW) architecture is widely adopted because it can explore the instruction level parallelism to achieve a high processing speed [10–13]. General VLIW architecture includes multiple special function units (FUs) that allow simultaneous execution for parallel operations [12,13]. Several VLIW architectures have been proposed based on the requirements of the specific applications, such as single register file VLIW architecture [10,14–17] and classical clustered VLIW architecture [10,18].

In practical application, most of the standard matrix size are less than or equal to 4×4 . A large matrix can be decomposed into the combination of these standard matrices. In an MIMO system, a 4×4 matrix is commonly used. In this paper, we propose a flexible VLIW architecture for the 4×4 matrix inversion. It has a global register file (RF) shared by every cluster. Every two adjacent clusters use a local RF. We also design a VLIW processor and its instruction sets. The proposed VLIW architecture can reduce the time of data exchange and gain a higher processing speed. Experimental results show that it takes 45 latency (cycles) to invert a 4×4 matrix, and only costs

Manuscript received December 24, 2012.

*Corresponding author.

This work was supported by the National Natural Science Foundation of China (61100155; 61227004; 61372071; 61372131; 61201289) and the Fundamental Research Funds of the Central Universities of China (K5051302096; JB140207).

0.30 μs when running at 150 MHz, which is faster than other solutions in processing speed.

The remainder of this paper is organized as follows: Section 2 introduces three classical matrix inversion algorithms. Section 3 gives the clustered VLIW architecture and its instructions. Then the performance of the proposed methods is analyzed in Section 4. Finally, Section 5 concludes this paper.

2. Matrix inversion algorithm

In the following subsections, three classical algorithms for matrix inversion are discussed. The complexity and application field of each algorithm is also analyzed in detail.

2.1 Direct analytic matrix inversion

The direct analytic approach is a straightforward way to compute the matrix inversion. The inversion of a 2×2 matrix is shown as follows:

$$H^{-1} = \begin{bmatrix} a & b \\ c & d \end{bmatrix}^{-1} = \frac{1}{ad-bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}. \quad (1)$$

This method is direct and easy to implement for the matrix with a small size. Unfortunately, the complexity of this method increases considerably for large matrices [7].

2.2 Matrix inversion based on QR decomposition

For a large matrix, the matrix inversion is traditionally implemented by applying QR factorization [4–6]. It can decompose the matrix M into an orthogonal matrix Q and an upper triangular matrix R , so that

$$M = QR. \quad (2)$$

Then, the inversion of M can be described as

$$M^{-1} = (QR)^{-1} = R^{-1}Q^{-1} = R^{-1}Q^T. \quad (3)$$

There are several ways to accomplish the QR decomposition, such as Givens rotation (GR) [5], squared Givens rotation (SGR) [5,19,20] and coordinate rotation digital computer (CRDC) [5,21]. However, all of these various inversion algorithms require square roots and division operations, which results in expensive hardware utilization. It is not worth inverting a small or middle size matrix by QR decomposition, because it occupies a considerable number of hardware resources.

2.3 Blockwise matrix inversion

As a state-of-art way for matrix inversion, the blockwise inversion is suitable for middle-sized matrices, e.g. a 4×4 matrix or slightly larger [1–2]. First, the original matrix M can be decomposed into four smaller sub-matrices

A, B, C and D as follows:

$$M = \begin{bmatrix} A & B \\ C & D \end{bmatrix} \quad (4)$$

where A is $n \times n$, D is $m \times m$. B and C are conformably dimensioned. If A is nonsingular, the inversion of M is defined as:

$$M^{-1} = \begin{bmatrix} E & F \\ G & H \end{bmatrix} \quad (5)$$

where

$$H = (D - CA^{-1}B)^{-1} \quad (6)$$

$$G = -(D - CA^{-1}B)^{-1}CA^{-1} \quad (7)$$

$$F = -A^{-1}B(D - CA^{-1}B)^{-1} \quad (8)$$

$$E = A^{-1} + A^{-1}B(D - CA^{-1}B)^{-1}CA^{-1} \quad (9)$$

The sub-matrices E, F, G, H in above equations can be obtained directly from the splitting of M . We can get the inversion of M by computing the inverse of two sub-matrices ($A, D - CA^{-1}B$) using the direct analytic approach and some additional matrix computations (e.g. matrix multiplication, matrix subtraction). It can be seen as a divide-and-conquer algorithm.

Obviously, the computation complexity of A and $D - CA^{-1}B$ affects the whole operation of inversion M . Since the direct analytic matrix inversion approach is a quick way for inverting the matrix with a small size, it is appropriate to split matrix M into smaller sub-matrices firstly and then applying the direct analytic approach to the small sub-matrices. By using this divide-and-conquer strategy, the blockwise matrix inversion is suitable for inverting middle-sized matrices. Considering the number of transmitted and received vectors in MIMO systems, the blockwise matrix inversion approach is used to invert a 4×4 matrix in this paper.

3. VLIW architecture for matrix inversion

3.1 VLIW architecture analysis

In the VLIW architecture, the FU is the executive part, while the RF is the data communication and data sharing part. There are two classical VLIW architectures [10,14–17]. They differ greatly in how the FUs visit the RFs. One of the VLIW architecture is shown in Fig. 1. In this architecture, all FUs access the same RF so that the data communication is convenient. This means that there is only one RF to store the large amount of data. Naturally, the major problem of this architecture is that it does not scale efficiently due to the long latency of RF and the instruction decoding logic. This shortcoming becomes the bottlenecks of area, power, and delay in a VLIW design [10].

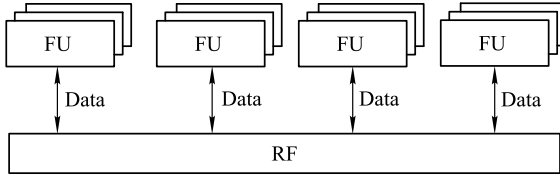


Fig. 1 Single RF VLIW architecture trajectory

In order to solve this problem, a VLIW architecture that partitions the single RF into uniform pieces, and organizes the processors with a classical clustered architecture to the corresponding local RFs is proposed, as shown in Fig. 2.

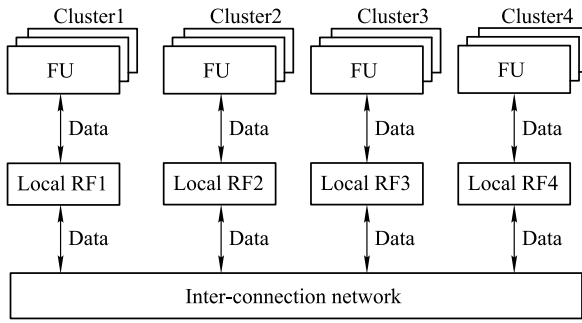


Fig. 2 Classical clustered VLIW architecture trajectory

Obviously, this architecture is more complex than the one in Fig. 1. In this clustered architecture, the function unit in a cluster can only access its own local RF, and all local RFs are connected by the inter-connection network. The performance of this architecture is mainly determined by the ability of data exchange between clusters. Thus, designing a suitable and efficient exchange network is the key issue of such architecture.

3.2 Proposed VLIW architecture for matrix inversion

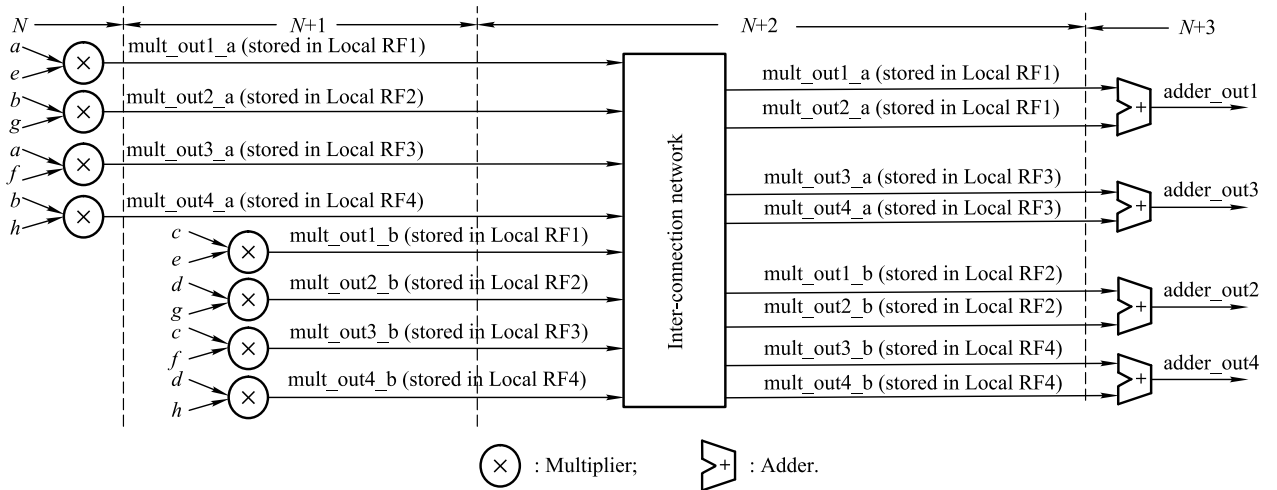
In Section 2, we pointed out that the sub-matrix (2×2 matrix) multiplication is one of the most fundamental operations in the process of inverting a 4×4 matrix by using the blockwise algorithm. The 2×2 matrix multiplication can be expressed as follows:

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} e & f \\ g & h \end{bmatrix} = \begin{bmatrix} ae + bg & af + bh \\ ce + dg & cf + dh \end{bmatrix} \quad (10)$$

where a, b, c, d, e, f, g, h are the elements of the two 2×2 sub-matrices. Fig. 3 shows the process of 2×2 matrix multiplication by using the clustered VLIW architecture in Fig. 2.

In Fig. 3, N is the time-slice of the pipeline. The multiplication results (e.g. mult_out1_a, mult_out2_a) are stored in the corresponding local RF (e.g. mult_out1_a is stored in local RF1) in the time-slice N and $N + 1$. In the time-slice of $N + 2$, data are exchanged through the inter-connection network. For example, mult_out2_a is stored in local RF2 initially. It is then moved to local RF1 after the exchange. In the time-slice $N+3$, the operand data of addition is ready. Each cluster performs addition operation simultaneously. Data exchange between cluster1 and cluster2, cluster3 and cluster4 occurs frequently during the process of 4×4 matrix inversion.

In order to reduce the time of data exchange and improve the processing speed, we can combine local RF1 and local RF2 into one local RF, and so do local RF3 and local RF4. Considering that some global data are usually used by all the FUs, a global RF is designed in the VLIW architecture to store the shared data. This improved architecture is shown in Fig. 4.

Fig. 3 Process of 2×2 matrix multiplication using classical clustered VLIW architecture trajectory

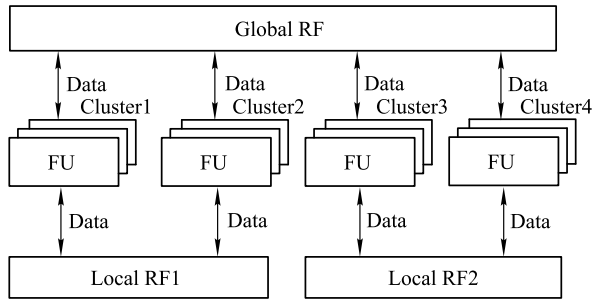


Fig. 4 Proposed new clustered VLIW architecture trajectory

In Fig. 4, cluster1 and cluster2 share a local RF1, while cluster3 and cluster4 share a local RF2. In this architecture, cluster1 can access the multiplication results of cluster2 directly to avoid the operation of data exchange. Based on this architecture, the processing of 2×2 matrix multiplication is shown in Fig. 5.

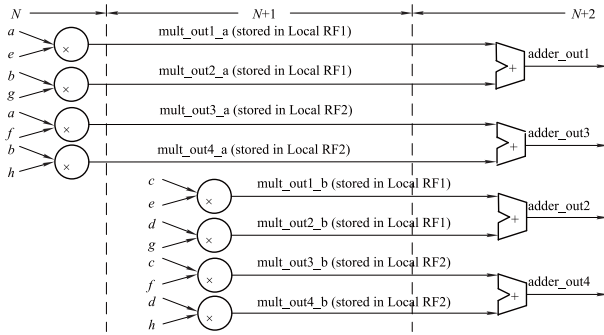


Fig. 5 Process of 2×2 matrix multiplication using new clustered VLIW architecture trajectory

3.3 VLIW processor

According to the architecture proposed above, the VLIW processor mainly includes four function units, a global RF and two local RFs. In the new VLIW processor, FUs can finish the arithmetic operation and the logical operation. Selection of RF size is very important because it affects the scale of the circuit. In Fig. 4, we assign $gr00, gr01, \dots, gr15$ to global RF, $gr16, gr17, \dots, gr23$ to local RF1 and $gr24, gr25, \dots, gr31$ to local RF2. In addition, some supplementary units of the processor are also indispensable. The detailed processor is shown in Fig. 6. The function of data and address generator (DAG) is generating address of the global RF and local RF. The addressing mode is based on indexed addressing. The indexed register and the modified register are designed to generate the physical address by adding both of them. The program sequencer (PS) is to schedule the procedure of the program that is loaded in the program memory. Two pages of input interface and output interface (I/O) caches are also designed to achieve the data exchange with the outside.

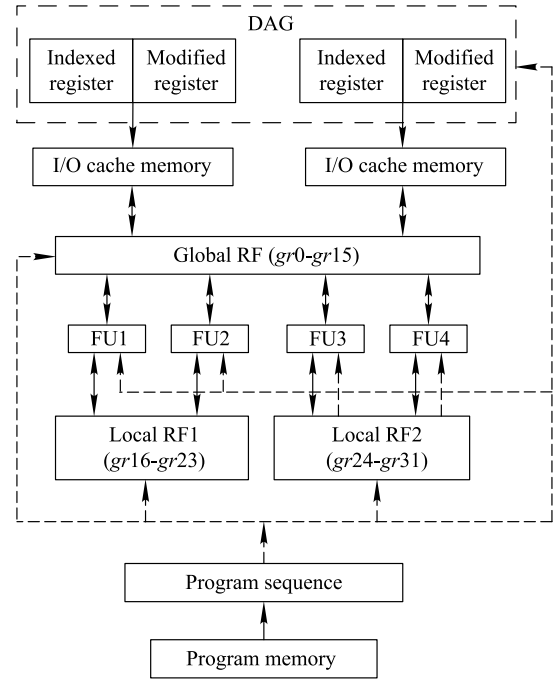


Fig. 6 VLIW processor trajectory

3.4 Instruction sets design and programming

In the VLIW processor, the instruction sets are closely correlated with the number of FUs. The instruction format is made up of four groups of sub-instructions. There are three kinds of sub-instruction sets: the operation instructions, the transfer instructions and the control instructions. The detailed functions of them are shown in Table 1.

Each of the sub-instructions occupies 12 bits binary codes. The former four bits indicate the function of this instruction. The next four bits are as the source operand. The rest of the four bits denotes the destination operand. There are four FUs in the VLIW processor. Thus, a full instruction has 48 bits binary codes to serve the processor.

With the proposed VLIW processor and its instruction sets, the following schedule gives an example of 2×2 matrix multiplication:

$$\begin{aligned}
 N : \quad & gr16 = gr00 \times gr04; \quad gr17 = gr01 \times gr06 \\
 & gr24 = gr00 \times gr05; \quad gr25 = gr01 \times gr07 \\
 N + 1 : \quad & gr18 = gr02 \times gr04; \quad gr19 = gr03 \times gr06 \\
 & gr26 = gr02 \times gr05; \quad gr27 = gr03 \times gr07 \\
 N + 2 : \quad & gr20 = gr16 + gr17; \quad gr21 = gr18 + gr19 \\
 & gr28 = gr24 + gr25; \quad gr29 = gr26 \times gr27
 \end{aligned} \tag{11}$$

where

$$[gr00 \ gr01 \ gr02 \ gr03 \ gr04 \ gr05 \ gr06 \ gr07] = [a \ b \ c \ d \ e \ f \ g \ h]. \tag{12}$$

Table 1 Sub-instruction sets

Name	Description
Operation instructions	Arithmetic operations: addition, subtraction, multiplication, accumulation, multiplication and accumulation (MAC) Logical operations: and, or, not, xor, shifting, bit-test
Transfer instructions	Ld (load data): load immediate, load data from the global RF or the local RFs, exchange data between the global RF and the local RFs, exchange the data inner of the global RF or the local RFs
Control instructions	Program control, conditional branch

In this example, the matrix multiplication only takes three clock cycles to complete an operation of 2×2 matrix multiplication. There is no need to accomplish the data exchange comparing with the classical clustered VLIW architecture. Therefore, it can reduce the clock cycles (latency) considerably when inverting a matrix, and improve the processing speed. In addition, the inversion of 2×2 matrix by using of the blockwise algorithm for the 4×4 matrix inversion can also be accomplished by this VLIW processor. The direct analytic matrix inversion method is a suitable way for the 2×2 matrix inversion. The division op-

eration in the direct analytic matrix inversion method can be achieved by the shifting instruction.

4. Performance analysis

Currently, most of the matrix inversion is implemented by DSP or FPGA platforms [1,8,9]. References [1,5] use the Virtex4 FPGA. Other implementation is based on the Virtex2 FPGA [8]. In this paper, we implement and synthesize the VLIW architecture based on the Virtex4 FPGA. The performance comparison of our implementation with other solutions is shown in Table 2.

Table 2 Implementation result of 4×4 matrix inversion

Items	Reference [1]	Reference [5]	Reference [8]	Reference [9]	Reference [23]	Our method
Implementation platform	FPGA (Virtex4)	FPGA (Virtex4)	FPGA (Virtex2)	DSP (TI'sC55x)	VLSI Processor	FPGA (Virtex4)
Number of logic slices	2094	9117	4400	N/A	N/A	3793
Number of DSP48	0	22	N/A	N/A	N/A	0
Latency (cycles)	120	933	175	269	200	45
Frequency /MHz	100	115	100	N/A	250	150
Time / μ s	1.20	8.11	1.75	1.59	0.80	0.30

In Table 2, the proposed VLIW architecture only takes 45 cycles (latency) to accomplish 4×4 matrix inversion. The frequency of this processor can work at 150 MHz. The total time for 4×4 matrix inversion is about 0.30 μ s. This is almost five times faster than the linear array proposed by [8], twenty times faster than the solutions proposed by [5], five times faster than the DSP (e.g. TI's C55x which is a classic VLSI processor) solution proposed by [9], and two times faster than the design-time configurable VLIW processor [23]. In addition, this new VLIW architecture can further add the FUs and connect them with the global RF and the local RFs easily. The instruction codes only need to add the new sub-instructions for the new FUs, following the original instruction codes. Thus, this architecture is flexible and scalable, and it can be easily extended to the matrix inversion in large sizes (e.g. 8×8 matrix) by loading corresponding programs.

5. Conclusions

In this paper, we propose a flexible and scalable VLIW architecture to accomplish the 4×4 matrix inversion. Experimental results show that our approach can reduce the

latency (cycles) considerably. Thus, it can get a high speed data throughout. The proposed VLIW architecture is programmable and is much more flexible than traditional FPGA. By loading corresponding programs, it can be easily modified to meet the needs of other applications in MIMO-OFDM systems.

References

- [1] J. Eilert, D. Wu, D. Liu. Efficient complex matrix inversion for MIMO software defined radio. *Proc. of the IEEE International Conference on Circuit and Systems*, 2007: 2610–2613.
- [2] L. Ma, K. Dickson, J. McAllister, et al. QR decomposition-based matrix inversion for high performance embedded MIMO receivers. *IEEE Trans. on Signal Processing*, 2011, 59(4): 1858–1867.
- [3] G. M. de Matos, H. C. Neto. On reconfigurable architectures for efficient matrix inversion. *Proc. of the IEEE International Conference on Field Programmable Logic and Applications*, 2006: 1–6.
- [4] C. K. Koc, G. Chen. Inversion of all principal submatrices of a matrix. *IEEE Trans. on Aerospace and Electronic Systems*, 1994, 30(1): 280–281.
- [5] M. Karkooti, J. Cavallaro, C. Dick. FPGA implementation of matrix inversion using QRD-RLS algorithm. *Proc. of the IEEE International Conference on Signals, Systems, and Computers*, 2005: 1625–1629.

- [6] C. K. Singh, S. H. Prasad, P. T. Balsara. VLSI architecture for matrix inversion using modified gram-schmidt based QR decomposition. *Proc. of the IEEE International Conference on VLSI Design*, 2007: 836–841.
- [7] D. Wu, J. Eilert, D. Liu, et al. Fast complex valued matrix inversion for multi-user STBC-MIMO decoding. *Proc. of the IEEE Computer Society Annual Symposium on VLSI*, 2007: 325–330.
- [8] F. Edman, V. Owall. A scalable pipelined complex valued matrix inversion architecture. *Proc. of the IEEE International Conference on Circuit and Systems*, 2005: 4489–4492.
- [9] P. Salmela, A. Happonen, T. Jarvinen, et al. DSP implementation of Cholesky decomposition. *Proc. of the IEEE Symposium on Trends in Communication*, 2006: 6–9.
- [10] Y. C. Zhao, C. J. Xue, M. M. Li, et al. Energy-aware register file re-partitioning for clustered VLIW architectures. *Proc. of the IEEE International Conference on Asia and South Pacific Design Automation*, 2009: 805–810.
- [11] Y. S. Lu, L. Shen, L. B. Huang. Optimal subgraph covering for customizable VLIW processor. *IET Computers and Digital Techniques*, 2009, 3(1): 14–23.
- [12] S. Rajagopalan, S. P. Rajan, S. Malik, et al. A retargetable VLIW compiler framework for DSPs with instruction-level parallelism. *IEEE Trans. on Computer-Aided Design of Integrated Circuit and Systems*, 2001, 20(11): 1319–1328.
- [13] S. J. Nam, Y. S. Kwon, Y. H. Im, et al. VLIW architecture with media instructions for image processing. *IEEE Trans. on Consumer Electronics*, 1999, 45(1): 192–202.
- [14] S. K. Chen, W. K. Fuchs. Compiler-assisted multiple instruction word retry for VLIW architectures. *IEEE Trans. on Parallel and Distributed Systems*, 2001, 12(12): 1293–1304.
- [15] C. H. Yu, K. Chung, D. Kim. An energy-efficient mobile vertex processor with multithread expanded VLIW architecture and vertex caches. *IEEE Journal of Solid-State Circuits*, 2007, 42(10): 2257–2269.
- [16] M. Sami, D. Sciuto, C. Silvano, et al. An instruction-level energy model for embedded VLIW architectures. *IEEE Trans. on Computer-Aided Design of Integrated Circuit and Systems*, 2002, 21(9): 998–1010.
- [17] A. K. Jones, R. Hoare, I. S. Kourtev, et al. A 64-way VLIW/SIMD FPGA architecture and design flow. *Proc. of the IEEE International Conference on Electronics, Circuits and Systems*, 2004: 499–502.
- [18] A. Wolfe, J. Fritts, S. Dutta. Datapath design for a VLIW video signal processor. *Proc. of the IEEE International Conference on High-Performance Computer Architecture*, 1997: 24–35.
- [19] R. Dohler. Squared givens rotation. *IMA Journal of Numerical Analysis*, 1991, 39(11): 1–5.
- [20] J. Gotze, U. Schwiigelshohn. A square root and division free givens rotation for solving least square problems on systolic arrays. *SIAM Journal on Scientific and Statistical Computing*, 1991, 12(7): 800–807.
- [21] J. Volder. The CORDIC trigonometric computing technique. *IRE Trans. on Electronic Computers*, 1959, 8(3): 330–334.
- [22] Z. Li, L. Fei. A research on an ASIP processing element architecture suitable for FPGA implementation. *Proc. of the Computer Science and Software Engineering*, 2008: 441–445.
- [23] S. Eberli, D. Cescato, W. Fichtner. Divide-and-conquer matrix inversion for linear MMSE detection in SDR MIMO receivers. *Proc. of the 26th Norchip Conference*, 2008: 162–167.

Biographies



Li Zhang was born in 1968. He received his M.S. degree from Xidian University in 1996. Currently, he is an associate professor in Xidian University. His research interests are image processing and VLSI design.
E-mail: zhangl_li2@mail.xidian.edu.cn



Fu Li was born in 1982. He got his B.S. and Ph.D. degrees from Xidian University in 2004 and 2010, respectively. From 2010 to 2011, he visited in the Microsoft Research Asia (MSRA). Currently, he is an associate professor in Xidian University. His research interests are nonlinear signal processing, HEVC architecture design and VLSI design.
E-mail: fuli@mail.xidian.edu.cn



Guangming Shi was born in 1965. He received his B.S. degree in automatic control, M.S. degree in computer control and Ph.D. degree in electronic information technology, all from Xidian University in 1985, 1988 and 2002, respectively. He joined the School of Electronic Engineering, Xidian University, in 1988. His research interests include compressed sensing, theory and design of multirate filter banks, image denoising, low-bit-rate image/video coding and implementation of algorithms for intelligent signal processing (using DSP&FPGA).
E-mail: gmshi@xidian.edu.cn